

Unit – 5 Database Triggers [5 Hrs]

A trigger is a named PL/SQL block stored in the Oracle Database and executed automatically when a triggering event takes place.

The event can be any of the following:

- A data manipulation language (DML) statement executed against a table e.g., INSERT, UPDATE, or DELETE. For example, if you define a trigger that fires before an INSERT statement on the customers table, the trigger will fire once before a new row is inserted into the customers table.
- A data definition language (DDL) statement executes e.g., CREATE or ALTER statement. These triggers are often used for auditing purposes to record changes of the schema.
- A system event such as startup or shutdown of the Oracle Database.
- A user event such as login or logout.

The act of executing a trigger is also known as firing a trigger. We say that the trigger is fired.

Benefits of Triggers

Triggers can be written for the following purposes –

- Generating some derived column values automatically
- Enforcing referential integrity
- Event logging and storing information on table access
- Auditing
- Synchronous replication of tables
- Imposing security authorizations
- Preventing invalid transactions

A trigger has two main parts: header and body.

The following illustrates the trigger header:

```
CREATE [OR REPLACE] TRIGGER trigger_name
  {BEFORE | AFTER } triggering_event ON table_name
  [FOR EACH ROW]
  [FOLLOWS | PRECEDES another_trigger]
  [ENABLE / DISABLE ]
  [WHEN condition]
```

And this is the trigger body:

```
DECLARE
  declaration statements
BEGIN
  executable statements
EXCEPTION
  exception_handling statements
END;
```

1) CREATE OR REPLACE

The CREATE keyword specifies that you are creating a new trigger. **The OR REPLACE keywords are optional. They are used to modify an existing trigger.**

Even though the OR REPLACE keywords are optional, they appear with the CREATE keyword in most cases.

2) Trigger name

Specify the name of the trigger that you want to create after the CREATE OR REPLACE keywords.

3) BEFORE | AFTER

The BEFORE or AFTER option specifies when the trigger fires, either before or after a triggering event e.g., INSERT, UPDATE, or DELETE.

4) ON table_name

The table_name is the name of the table associated with the trigger.

5) FOR EACH ROW

The clause FOR EACH ROW specifies that the trigger is a **row-level trigger**. A row-level trigger fires once for each row inserted, updated, or deleted.

6) ENABLE / DISABLE

The ENABLE / DISABLE option specifies whether the trigger is created in the **enabled** or **disabled** state. Note that if a trigger is disabled, it is not fired when the triggering event occurs.

By default, if you don't specify the clause ENABLE / DISABLE, the trigger is created with the **enabled** state.

7) FOLLOWS | PRECEDES another_trigger

For each triggering event e.g., INSERT, UPDATE, or DELETE, you can define multiple triggers to fire. In this case, you need to specify the firing sequence using the FOLLOWS or PRECEDES option.

BEFORE and AFTER Triggers:

Triggers can run BEFORE the action is taken or AFTER the action is taken.

A **BEFORE Trigger** means that Oracle will fire this trigger before the Insert, update or delete operation is executed.

BEFORE triggers are usually used when validation needs to take place before accepting the change. They run before any change is made to the database. Let's say you run a database for a bank. You have a table accounts and a table transactions. If a user makes a withdrawal from his account, you would want to make sure that the user has enough credits in his account for his withdrawal. The BEFORE trigger will allow to do that and prevent the row from being inserted in transactions if the balance in accounts is not enough.

AFTER TRIGGERS are used when the triggering statements should completed before executing the trigger action.

AFTER triggers are usually used when information needs to be updated in a separate table due to a change. They run after changes have been made to the database (not necessarily committed). Let's go back to our back example. After a successful transaction, you would want balance to be updated in the accounts table. An AFTER trigger will allow you to do exactly that.

Syntax of BEFORE Trigger:

```
CREATE [ OR REPLACE ] TRIGGER trigger_name
BEFORE INSERT OR UPDATE OR DELETE
    ON table_name
    [ FOR EACH ROW ]

DECLARE
    -- variable declarations

BEGIN
    -- trigger code

EXCEPTION
    WHEN ...
    -- exception handling

END;
```

Syntax of AFTER Trigger:

```
CREATE [ OR REPLACE ] TRIGGER trigger_name
AFTER INSERT OR UPDATE OR DELETE
    ON table_name
    [ FOR EACH ROW ]

DECLARE
    -- variable declarations

BEGIN
    -- trigger code

EXCEPTION
    WHEN ...
    -- exception handling

END;
```

Example of BEFORE Trigger:

Consider the following **employee** table:

EID	NAME	SALARY
101	Ram	25000
102	Shyam	55000
103	Gita	34000

Example 1

Let's create a Trigger which simply display a message before inserting, updating and deleting a row in table.

```
CREATE OR REPLACE TRIGGER mytrigger
BEFORE INSERT OR UPDATE OR DELETE
ON employee
FOR EACH ROW

BEGIN
    DBMS_OUTPUT.PUT_LINE('Trigger Test!');
END;
/
```

Above Trigger will be fired while inserting new data as follows:

```
INSERT INTO employee VALUES(101,'Raaju',30000);
```

Output:

```
1 row(s) inserted.
Trigger Test!
```

Example 2

```
CREATE OR REPLACE TRIGGER mytrigger
BEFORE INSERT OR UPDATE OR DELETE
ON employee
FOR EACH ROW
WHEN (new.salary>30000) --condition

BEGIN
    DBMS_OUTPUT.PUT_LINE('Trigger Test!');
END;
/
```

Above Trigger will not be fired while inserting new data if salary is less than 30000.

```
INSERT INTO employee VALUES(101,'Raaju',29000);
```

Output:

```
1 row(s) inserted.
```

Above Trigger will be fired while inserting new data if salary is greater than 30000:

```
INSERT INTO employee VALUES(101, 'Raaju', 32000);
```

Output:

1 row(s) inserted.

Trigger Test!

Example – 3

Let's create a trigger that **display's difference between new salary and old salary of employee while updating salary of employee.**

```
CREATE OR REPLACE TRIGGER mytrigger
BEFORE UPDATE
ON employee
FOR EACH ROW

DECLARE
    sal_diff NUMBER;
BEGIN
    sal_diff:= :new.salary - :old.salary;
    DBMS_OUTPUT.PUT_LINE('New salary is:' || :new.salary);
    DBMS_OUTPUT.PUT_LINE('Old salary is:' || :old.salary);
    DBMS_OUTPUT.PUT_LINE('Difference is:' || sal_diff);
END;
/
```

Now, Let's execute following update query:

```
UPDATE employee SET salary=salary+5000
WHERE eid=103;
```

Output:

1 row(s) updated.

New salary is:39000

Old salary is:34000

Difference is:5000

:NEW and :OLD Clause

In a row level trigger, the trigger fires for each related row. And sometimes it is required to know the value before and after the DML statement.

Oracle has provided two clauses in the RECORD-level trigger to hold these values. We can use these clauses to refer to the old and new values inside the trigger body.

- **:NEW** – It holds a new value for the columns of the base table/view during the trigger execution
- **:OLD** – It holds old value of the columns of the base table/view during the trigger execution

This table illustrates the availability of :NEW and :OLD variables by the triggering event:

Triggering Event	:NEW	:OLD
INSERT	Yes	No
UPDATE	Yes	Yes
DELETE	No	yes

Example of AFTER Trigger

```
CREATE OR REPLACE TRIGGER mytrigger
AFTER INSERT OR DELETE
ON employee
FOR EACH ROW

BEGIN
    DBMS_OUTPUT.PUT_LINE('After Trigger Executed!');
END;
/
```

Now, execute delete query as follows:

```
DELETE FROM employee WHERE eid=102;
```

Output:

```
1 row(s) deleted.
After Trigger Executed!
```

Row and Statement Triggers

A row-level trigger fires once for each row that is affected by a triggering event. For example, if deletion is defined as a triggering event for a particular table, and a single DELETE statement deletes five rows from that table, the trigger fires five times, once for each row.

Examples written above are the examples of Row triggers.

A statement-level trigger fires only once for each statement. Using the previous example, if deletion is defined as a triggering event for a particular table, and a single DELETE statement deletes five rows from that table, the trigger fires once.

A statement-level trigger is fired whenever a trigger event occurs on a table regardless of how many rows are affected. In other words, a statement-level trigger executes once for each transaction.

For example, if you update 1000 rows in a table, then a statement-level trigger on that table would only be executed once.

Due to its features, a statement-level trigger is not often used for data-related activities like auditing the changes of the data in the associated table. It's typically used to enforce extra security measures on the kind of transaction that may be performed on a table.

By default, the statement CREATE TRIGGER creates a statement-level trigger when you omit the FOR EACH ROW clause.

Syntax of Statement Trigger:

```
CREATE [ OR REPLACE ] TRIGGER trigger_name
AFTER INSERT OR UPDATE OR DELETE
    ON table_name

DECLARE
    -- variable declarations

BEGIN
    -- trigger code

EXCEPTION
    WHEN ...
    -- exception handling

END;
```

Difference between Row and Statement Triggers

Row Level Triggers	Statement Level Triggers
Row level triggers executes once for each and every row in the transaction.	Statement level triggers executes only once for each single transaction.
Specifically used for data auditing purpose.	Used for enforcing all additional security on the transactions performed on the table.
“FOR EACH ROW” clause is present in CREATE TRIGGER command.	“FOR EACH ROW” clause is omitted in CREATE TRIGGER command.
Example: If 1500 rows are to be inserted into a table, the row level trigger would execute 1500 times.	Example: If 1500 rows are to be inserted into a table, the statement level trigger would execute only once.

INSTEAD OF Triggers

INSTEAD OF triggers provide a way to modify views that cannot be modified directly through DML statements like INSERT, UPDATE and DELETE.

An INSTEAD OF trigger is

1. Always a row-level trigger.
2. Can read OLD and NEW values, but cannot change them.
3. Cannot be conditional. Means we can not add WHEN or IF condition.

If the view has an INSTEAD OF trigger, it will automatically skip the DML statement and execute other DML statements instead.

Note that an INSTEAD OF trigger is fired for each row of the view that gets modified.

In Oracle, you can create an INSTEAD OF trigger for a view only. You cannot create an INSTEAD OF trigger for a table.

Syntax of creating an INSTEAD OF trigger:

```
CREATE [OR REPLACE] TRIGGER trigger_name
INSTEAD OF {INSERT | UPDATE | DELETE}
ON view_name
FOR EACH ROW
BEGIN
    EXCEPTION
    ...
END;
```

Example of INSEAD OF Trigger

Let's create two tables named as **Employee** and **Department**.

```
CREATE TABLE employee(eid INT, name VARCHAR2(30), dept_id INT);
CREATE TABLE department(dept_id INT,dept_name VARCHAR2(30));
```

Now let's create a **view** named as **vw_employees** containing records from **employee** and **department** both tables.

```
CREATE VIEW vw_employees AS
SELECT eid,name,dept_id,dept_name
FROM employee
INNER JOIN department
USING (dept_id);
```

Now, let's try to insert data in above view, you'll get following error:

```
INSERT INTO vw_employees(eid,name,dept_id,dept_name)
VALUES(101,'Ram',001,'Finance');
```

Output:

ORA-01779: cannot modify a column which maps to a non key-preserved table

Above error can be solved using INSTEAD OF Trigger.

Now let's create INSTEAD OF Trigger to insert data in above view as follows:

```
CREATE OR REPLACE TRIGGER mytrigger
INSTEAD OF INSERT ON vw_employees
FOR EACH ROW
BEGIN
    --inserting into employee table
    INSERT INTO employee(eid,name,dept_id)
    VALUES(:new.eid,:new.name,:new.dept_id);

    --inserting into department table
    INSERT INTO department(dept_id,dept_name)
    VALUES(:new.dept_id,:new.dept_name);
END;
```


On successful execution, this trigger will insert a new row of data into both the underlying tables of the **view vw_employees (Employee and Department)**. You can confirm that by executing an insert DML over the view.

Let's try to insert data in a view **vw_employees** as follows:

```
INSERT INTO vw_employees(eid,name,dept_id,dept_name)
VALUES
(101,'Ram',005,'Finance');
```

Now select Employee and Department table you'll see following output:

```
SELECT * FROM employee;
```

EID	NAME	DEPT_ID
101	Ram	5

```
SELECT * FROM department;
```

DEPT_ID	DEPT_NAME
5	Finance